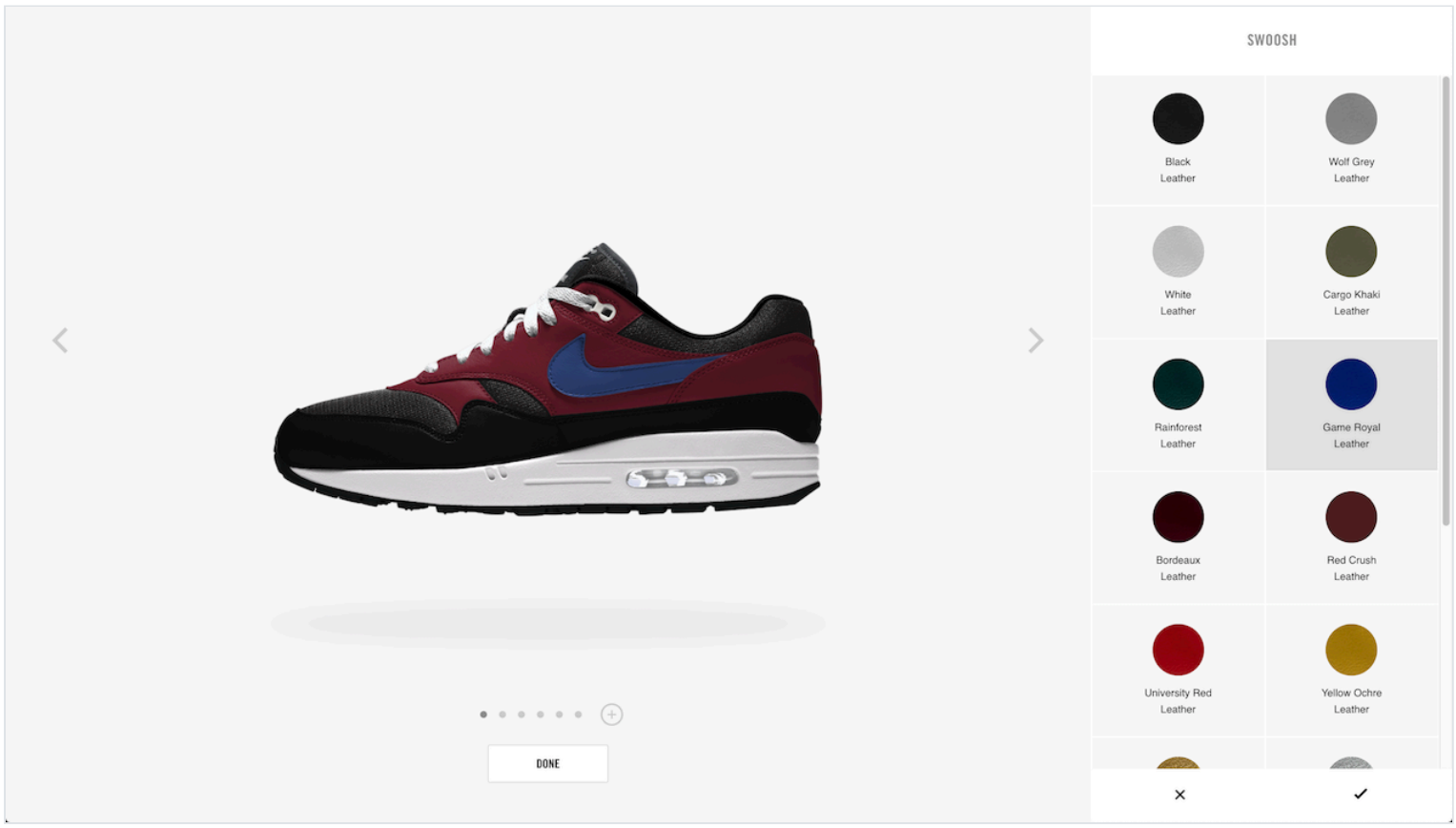
ADDING CUSTOMIZATION TO YOUR EXPERIENCE

TABLE OF CONTENTS

Adding Customization to Your Experience

Last Updated: 06/01/2022

The **Customization Experience Platform (CXP)** unlocks your ability to add premium product customization features to your experience, similar to Nike By You:



> **TIP**: Before using this guide, you should have already completed Customization Overview.

Introduction

In this guide, we will discuss how to integrate CXP customization features into your app. First, what does CXP have to offer?

Builder Bundle

The Builder Bundle is a JavaScript bundle (known as B16) that is your main interface with CXP.

Users of B16 have two adventure options:

- **Full Bundle**: Includes access to the fully-styled UX for customizing products
- **Headless or Builder API**: Allows you to interact with Builder methods and associated JS APIs where customization business logic is housed so that you can build your own experience.

REST APIs

In addition to the Builder, the Customization Domain offers a variety of REST APIs that can be used for specific steps along the user journey.

Table 1: Customization APIs and What They Do

Service Name	What does it do?
Consumer Designs	Returns the entire payload of a consumer design for rendering in experiences.
Design View	Returns a subset of consumer design data for rendering in experiences.

Service Name	What does it do?
Image Redirect (for a consumer design)	Returns a full Scene7 Render URL to display a consumer's design image. Also includes optional parameters for image view, size, and redirect proxy (for legacy clients).
Bill of Materials	Returns the factory-facing design elements of the consumer's design.
Inspiration Designs	Returns the entire payload of an inspiration design for rendering in experiences.

Key Terms

Table 2: Key Customization Terms

Term	Definition
Builders	A configuration ("concept") of a customizable product that contains all possible variations of questions, answers, materials, colors, etc. Represented by a `pathName` identifier.
B16	The JavaScript bundle that contains the customization UX and Builder API.
Builder Components	Consumer-facing options, the visual choices and selections.
Builder Data	Data snapshot of the Builder as returned by the Builder API, including the consumer's gender, width, size selections, pricing, and more.
Questions & Answers	A programmatic way of determining the consumer's selections and how they map to the various pieces of the shoe. Often new answers result in visual changes within the Builder.
Design	A locked set of Builder data that maps to specific selections made by the consumer.
Design ID	(Formerly known as metric ID). A reference to the user's design that can be used to call other Customization Services to return key design data and factory-facing bill of materials.
PathName	An alphanumeric unique identifier that is used to reference a specific Builder.
Prebuild	A set of pre-determined Builder data used for merchandising and to engage consumers in the design/ buying experience. Prebuilds are not purchasable until selecting a size.

Quick-Start: Load the Builder

Want to load the Builder bundle and start interacting with a basic Customization experience? Read this section, otherwise skip to Show Customizable Products.

In your app's source, create an HTML template and follow these steps:

1. **Include the Builder bundle**

   Add a `<script>` tag in the `<body>` to include the Builder JavaScript bundle like:

   ```
   <script src="https://assets.commerce.nikecloud.com/nikeid/builder/dist/
   b16Builder.bundle.min.js" type="text/javascript"></script>
   ```

2. **Add a for the Builder to load into**

Add a `<div>` in the `<body>` with an `id="nikeid-app"` attribute like:

```
<div id="nikeid-app-container" style="width: 1000px; height: 800px;">
    <div id="nikeid-app">
    </div>
</div>
```

3. **Load the Builder**

   Add a `<script>` tag in the `<body>` that invokes the `nikeIdBuilder(rootElement, config)` function like:

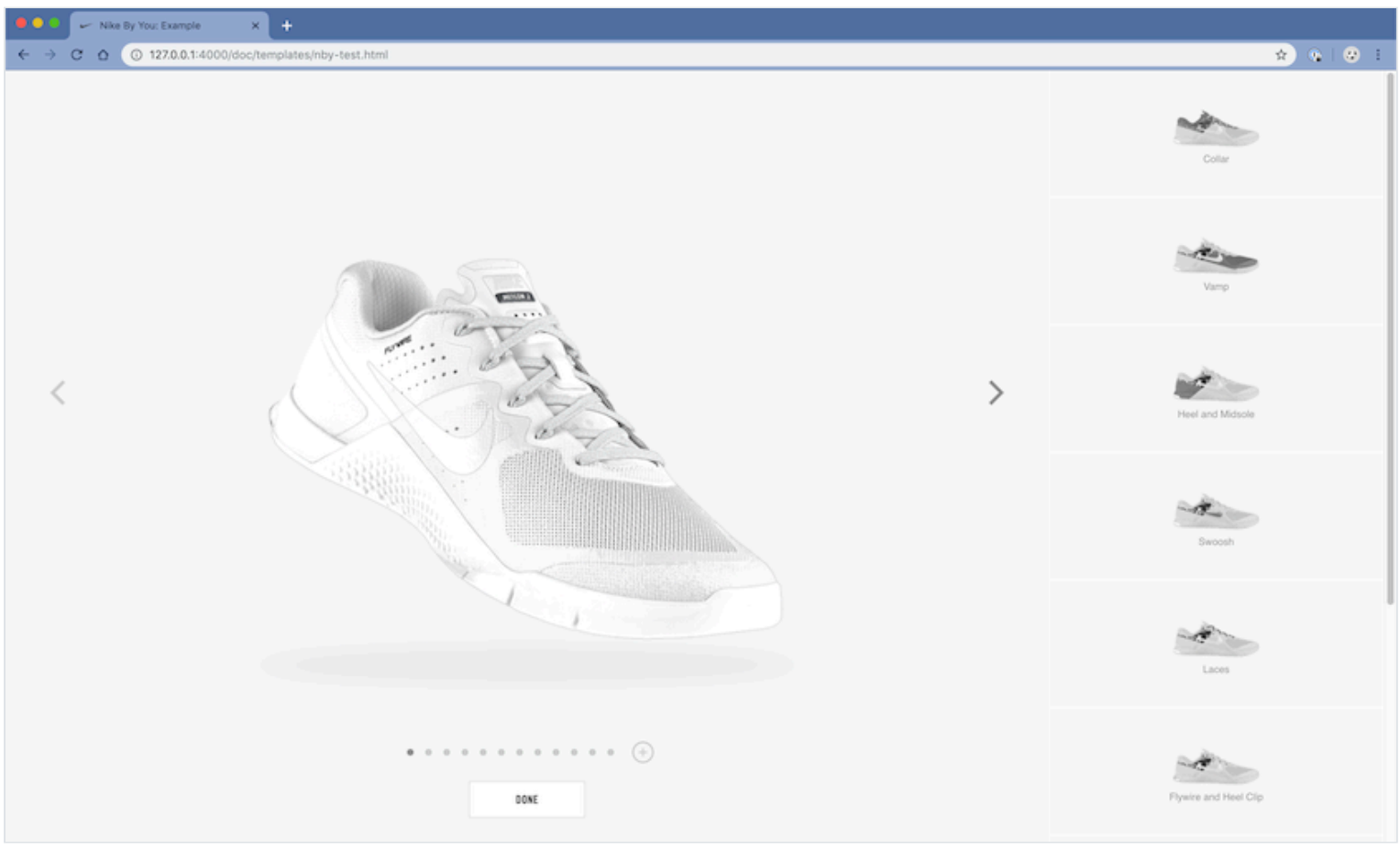
```
<script>
  const builderApi = nikeIdBuilder(document.getElementById('nikeid-app'), {
    'nike-api-caller-id': 'com.nike:commerce.b16.localhost',
    pathName: 'KobeAD2exoFA18'
  })
</script>
```

   TIPS:

   - The argument for the `rootElement` parameter can be populated with a method like `document.getElementbyId('element-id-where-builder-renders')`.
   - The argument for the `config` parameter must contain at minimum the `nike-api-caller-id` and `pathName` properties.
   - See Customization Builder Reference for details about the Builder.

4. **Navigate to Your Local Host to View the Builder Experience**

   - The Builder loads into your chosen HTML element (in this case, a `<div>` with attribute `id="nikeid-app"`), and it invokes the necessary services to render the experience:
   - Browse to the HTML page on your localhost to view the Builder experience. The URL will vary depending on how your app is being served up locally.



Example HTML template (initializes builder only):

```
 <!DOCTYPE html>
 <html>
 <head>
     <title>Nike By You: Example</title>
     <meta charset="utf-8">
     <meta http-equiv="cache-control" content="max-age=0,no-cache,no-store"/>
     <meta http-equiv="expires" content="Tue, 01 Jan 1980 1:00:00 GMT"/>
     <meta http-equiv="pragma" content="no-cache,no-store"/>
     <meta name="viewport"
           content="width=device-width, initial-scale=1.0, minimum-scale=1.0, maximum-
scale=1.0, user-scalable=no, viewport-fit=cover"/>
     <link rel="icon" href="//www.nike.com/favicon.ico"/>
     <script type="text/javascript">
     var nsgConfig = {
       HOST: '//www.nike.com',
       PLATFORM: 'mobile'
     };
     </script>
     <script type="text/javascript" src="//www.nike.com/styleguide/init/
nsg.js"></script>
 </head>
 <body>
 <div id="nikeid-app-container" style="width: 1000px; height: 800px;">
     <div id="nikeid-app">
     </div>
 </div>
 <script type="application/javascript"
         src="https://assets.commerce.nikecloud.com/nikeid/builder/dist/
b16Builder.bundle.min.js"></script>
 <script>
   const builderApi = nikeIdBuilder(document.getElementById('nikeid-app'), {
     'nike-api-caller-id': 'com.nike:commerce.b16.localhost',
     pathName: 'KobeAD2exoFA18',
   })
 </script>
 </body>
 </html>
```

> **TIP**: See more at Customization Builder Reference, which is the single source of truth for Builder functionality.

Show Customizable Products

Which products are customizable? How do I start designing?

Step 1: Load the Builder

Load the Builder and interact with the API to help drive the product browsing experience. Later, you can Show a Design Experience and Enable Purchasing without having to first load the Builder.

Load the Builder by invoking the `nikeIdBuilder(rootElement, config)` function.

- In the `pathName` property of the `config` argument, use the value in `objects.productInfo.customizedPreBuild.legacy.pathName` from the Product Feeds response, like `pathName: 'af1LowChampsSU19'`.
- Use `bridge`, a property of the `config` parameter, to listen for events coming back from the Builder.

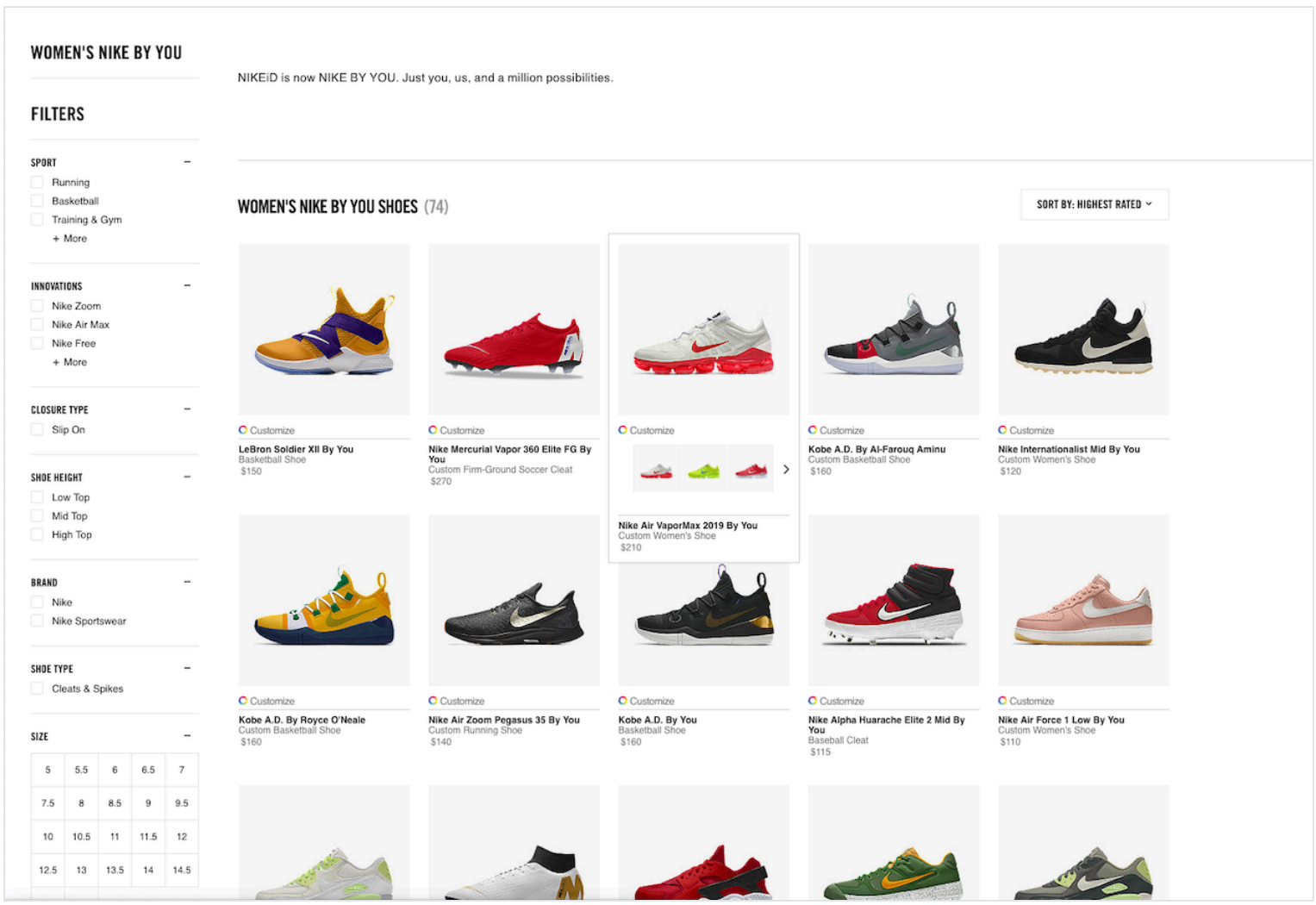
  Example (loads the Builder and logs a few things):

```
  <script>
  const builderApi = nikeIdBuilder(document.getElementById('nikeid-app'), {
    'nike-api-caller-id': 'com.nike:commerce.b16.localhost',
    pathName: 'FUTUREELITEFA18',
    fetchCapacity: true,
    sizeTypeRegion: 'US',
    bridge: {
      onProductLoad: function(buildData) {
      // Builder is ready
      console.log('Set Size
Answer',builderApi.setSizeAnswer('FUTUREELITEFA18:LTITEM8538:LTITEM8112:LTITEM8010:
LTITEM403108','LTITEM8132','us-mens'));
      },
      onError: function(error) {
        console.error(error);
      },
      onDone: function(buildData) {
      // Consumer selected Done button
        console.log('Done:',buildData);
      },
      onPriceUpdate: function(priceData) {
      // Customization selections have changed the product price
        console.log('Price Update:',priceData);
      }
    }
  })
  </script>
```

> **TIP**: See Quick-Start: Load the Builder and Customization Builder Reference for more details about the Builder.

Step 2: Show Customizable Products & Color Options

Whether it's a Nike By You web experience with it's product grid walls and Product Detail Pages (PDPs), or some other type of experience, you need to show the consumer which products, and in what colors, can be customized.



Get a list of customizable products, along with relevant content.

- Call either the Product Feeds API or the Rollup Threads API
- To select only 'Nike By You' products, use the `filter=attributeIds()` query parameter like:

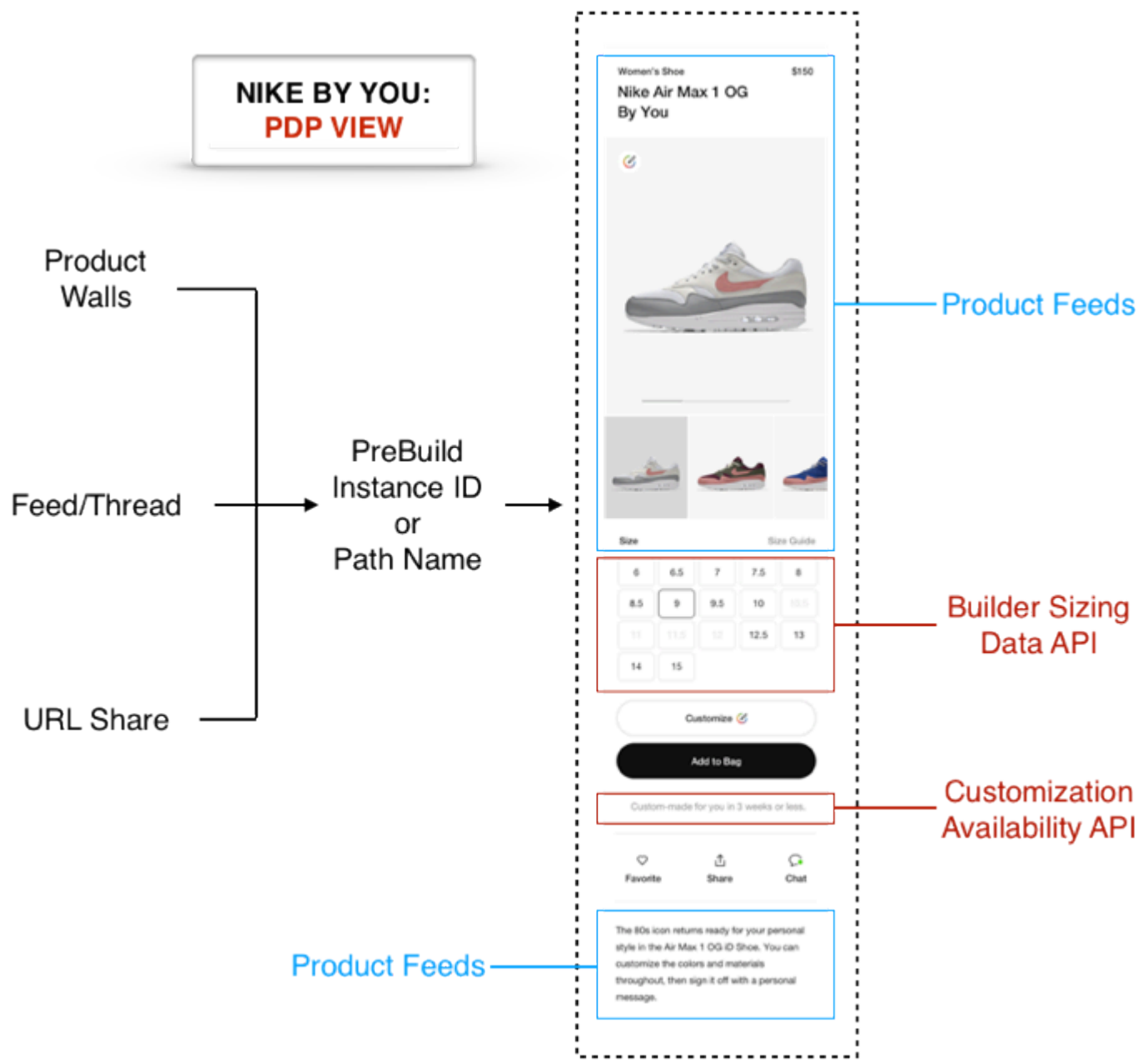
  https://api.nike.com/product_feed/threads/v2?filter=channelId(d9a5bc42-4b9c-4976-858a-f159cf99c647)&filter=marketplace(US)&filter=language(en)&filter=attributeIds(92be6a0f-24dd-4e2e-87d0-5ce4ade3a923.

Use the response data to drive the experience of browsing customizable products (grid wall, feed, etc.).

> **TIPS**:
>
> - Rollup Threads is best for displaying a grid wall, where alternate colors are shown with each product in the grid.
> - See Adding Rollup Threads to Your Experience and Adding Product Feeds to Your Experience for more integration info.

Step 3: Show a PDP for a Customizable Product

Show the consumer a particular product in more detail with a PDP like:



Next, we'll discuss some components that you can include in a PDP.

Step 3a: Show Product Content and Info

Show the product images, pricing, and other info from Product Feeds on the PDP.

- Call the Product Feeds API with the thread id to retrieve content and info for the product.

Step 3b: Show 'Edit Design' CTA

Show the consumer a way to edit the design.

Display an 'Edit Design' CTA (call-to-action) button that launches the Builder UX

- Example button code using NCSS:

```
<button class="ncss-btn-primary-dark">Edit Design</button>
```

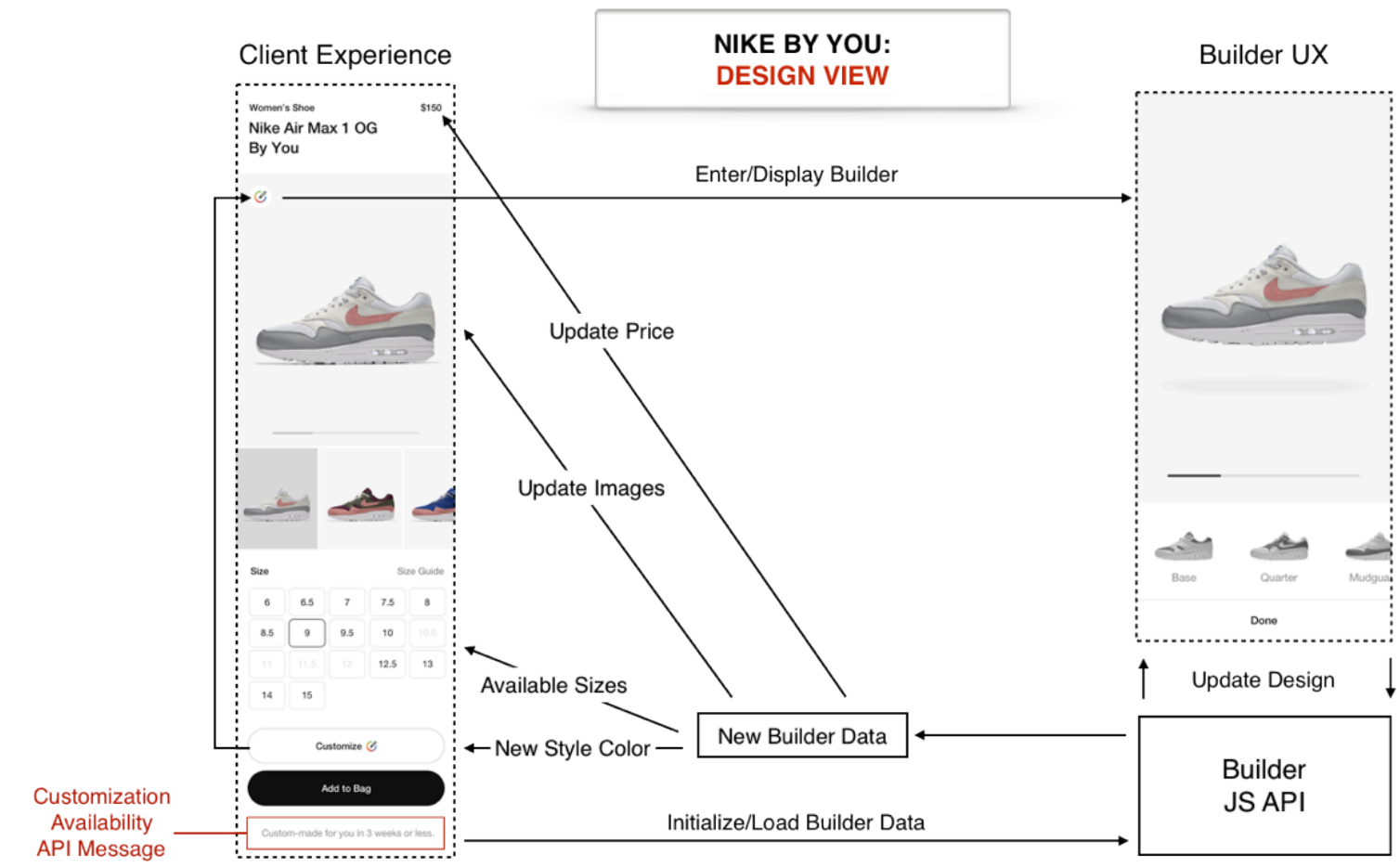
Edit Design

> **TIP**: It's recommended for the 'Edit Design' CTA to be always active on the PDP.

Show a Design Experience

What customization options are available? What does my design look like? How much will it cost?

The consumer has selected to edit the design via the 'Edit Design' CTA, so it's time to show them the Builder UX.



Show the HTMLElement containing the Builder UX

- Use same Element ID you used when loading the Builder, e.g. `document.getElementById('nikeid-app')`.

Interact with the Builder

Table 3: Builder Scenarios with Corresponding UX Interactions

Scenario	Interaction
Load a new build, either to "reset" the builder or to switch between builds.	Invoke the `setBuild` method, for example by prebuild ID or metric ID.
Display price changes (when customization options are changed).	Listen to the `onPriceUpdate(priceData)` bridge callback and show updated price in UX.
Consumer selects the 'Done' button.	Listen to the `onDone(buildData)` bridge callback, then call `saveDesign` and update UX.
Save a build.	Invoke the `saveDesign` method, which returns a metric ID for the build.
Edit a design that is already in the cart.	Invoke `setBuild` with the metric ID you previously got from calling `saveDesign`.
A consumer triggers an analytics event.	Listen to the `onAnalyticsEvent(type, payload)` bridge callback, then trigger an action.
An error occurs in the Builder.	Listen to the `onError(error)` bridge callback, handle the error and update UX.

> **TIP**: See Bridge Properties for more.

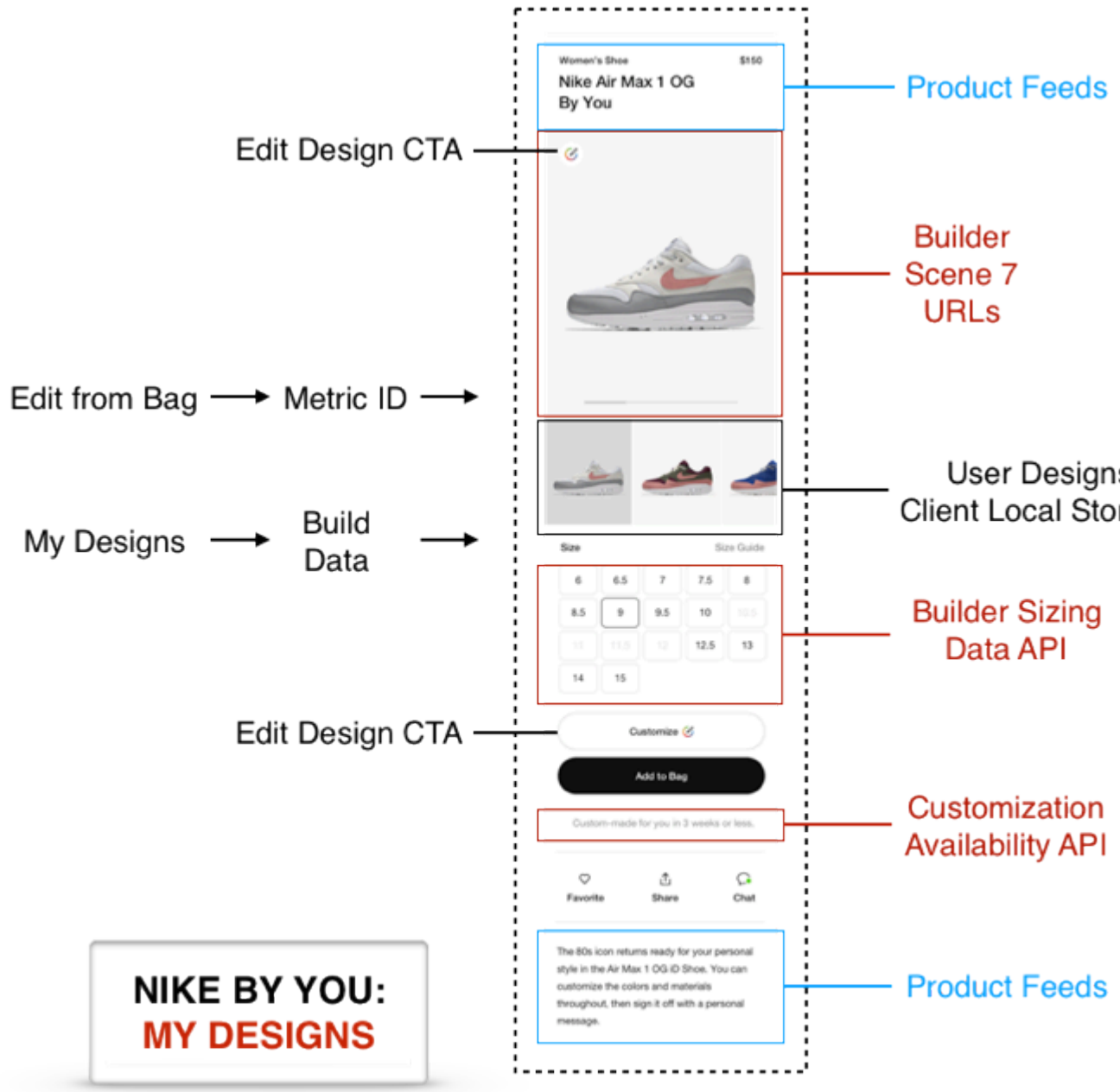
Sample JavaScript

For a sample JavaScript class that shows how you might interact with the Builder, see builderBridge.js.

Enable My Designs

How do I save a design in My Designs?

The consumer may wish to save one or more of their designs for later in My Designs.



1. **Enable My Designs**

   Add the following properties to the configuration object you are passing into the `nikeIdBuilder` function, like:

```
const config = {
 myDesignsEnabled: true,
 myDesignCapacity: 20
};
```

These two `config` properties are described below in more detail:

Property Name	Usage	Default
`myDesignsEnabled`	Value of `true` tells the Builder to enable myDesign local storage, while `false` disables it	`false`
`myDesignsCapacity`	Number of most recent designs to be stored in local storage. If this maximum value is exceeded, the oldest design will be deleted from storage.	`15`

Once My Designs is enabled, when the user clicks the ‘Done’ button their design will be saved to the myDesigns local storage.

2. **Get a list of the consumer’s My Designs**

   Call any of the following Builder or bridge methods:

   - setAnswer
   - setSizeType
   - setSizeAnswer
   - OnProductLoad
   - OnDone
   - getMyDesigns

   In all cases, the build data that is returned to your application includes a list of the myDesigns that have been stored for the current `pathName`, sorted from newest to oldest, like:

```
 myDesigns: [
    {
       imgUrl: "http://render.nikeid.com/ir/render/nikeidrender/
AMax20171611_v9?obj=/s/shadow/shad&show&color=000000&obj=/s/
g1&color=3a3a3a&show&obj=/s/g4&color=3a3a3a&show&obj=/s/g7&color=141414&show&obj=/s/
g8&color=ffffff&show&obj=/s/g9&color=141414&show&obj=/s/g6&color=ffffff&show&obj=/s/
g14&color=141414&show&obj=/s/g13&color=141414&show&obj=/s/
g15&color=bcc6cc&show&obj=/s/g2&color=ffffff&show&obj=/s/g5/
solid&color=141414&show&obj=/s/g10/solid&color=b7132d&show&obj=/s/g12/
solid&color=141414&show&obj=/s/g17/solid&color=ffffff&show&obj=/s/
g18&color=ffffff&show&obj=/s/g23&color=000001&show&obj=/s&req=object&fmt=png-
alpha&icc=AdobeRGB&wid=250"
       key: "1561569449311"
       pathName: "AMax20171611_GLOW"
    },
    {
       imgUrl: "http://render.nikeid.com/ir/render/nikeidrender/
AMax20171611_v9?obj=/s/shadow/shad&show&color=000000&obj=/s/
g1&color=3a3a3a&show&obj=/s/g4&color=3a3a3a&show&obj=/s/g7&color=141414&show&obj=/s/
g8&color=ffffff&show&obj=/s/g9&color=141414&show&obj=/s/g6&color=ffffff&show&obj=/s/
g14&color=141414&show&obj=/s/g13&color=141414&show&obj=/s/
g15&color=bcc6cc&show&obj=/s/g2&color=ffffff&show&obj=/s/g5/
solid&color=141414&show&obj=/s/g10/solid&color=154399&show&obj=/s/g12/
solid&color=141414&show&obj=/s/g17/solid&color=ffffff&show&obj=/s/
g18&color=ffffff&show&obj=/s/g23&color=000001&show&obj=/s&req=object&fmt=png-
alpha&icc=AdobeRGB&wid=250"
       key: "1561569431459"
       pathName: "AMax20171611_GLOW"
    }
 ]
```

   The fields in myDesigns are described below:

Field	Description	Type
`imgUrl`	A Scene7 url for view9 of the myDesign thumbnail	String
`key`	The unique key associated with the myDesign	String
`pathName`	The pathName associated with the myDesign	String

3. **Manage a consumer’s My Designs**

   Perform actions on My Designs by calling the following methods:

   - To load a My Design: applyMyDesign(myDesignKey)
   - To save a My Design: saveMyDesign()
   - To delete a My Design: deleteMyDesign(myDesignKey)

- To delete all My Designs: deleteMyDesigns()

Enable Purchasing

Is the product available for purchase? How do I select a size? When would my design be delivered to me?

The consumer is finished customizing their product, so it's time to get them ready for the checkout process.

Show Product Availability Messaging

Show the consumer on the PDP whether the product can be purchased. If it can, show an estimated lead time (in weeks) for the product to be delivered. Here a few example messages:

Table 4: Product Availability Conditions with Corresponding PDP Message

Condition	Message on PDP
Product is available	`"Custom-made and delivered to you in 4 weeks or less."`
Product is not available	`"The product is currently unavailable."`

To retrieve the availability and lead-time data, there are two options:

Call the Customization Availability API

- The API returns availability by size, lead-time in days, and message text, all for a particular style-color.
- In the `pathName` query parameter, use the value in `objects.productInfo.customizedPreBuild.legacy.pathName` from the Product Feeds API response, like `https://api.nike.com/customization/availability/v1/us/en_US?filter=pathName(af1LowChampsSU19)`.

OR

Read the Build Data

- Availability: From the returned Build Data, if `sizingData.displayName` is "Size", then loop through `sizingData.answers` and evaluate whether `isAvailable` is true or false for all sizes.
- Lead Time: Call the getLeadTimeMessage method of the Builder API to get the message text and lead time in days for the product.

> **TIP**: Remember, by initializing and interacting with the Builder API prior to showing the Builder UX, you can access the Build Data. See Step 1: Load the Builder for more.

Show Gender, Width and Size Options and Confirm Consumer's Selections

Show the consumer all of the possible gender and size options for the product. From the possible sizes, show which sizes are available for purchase. Allow the consumer to make their gender and size selections.

Show gender options (if applicable), and confirm consumer's selection

- Use the info from the `sizingData` object (in the Build Data) to display the available genders, making note of the respective `questionId` and `answerId` values.
- Using the `questionId` and `answerId` values for the gender selected by the consumer, call the `setAnswer` method of the Builder API, like:

```
builderApi.setAnswer('ER2teamSP19_barca:LTITEM8538:LTITEM8112','LTITEM8011','')`
```

  This sets `isSelected: true` in `sizingData.answers`, indicating that a particular gender was selected.

- Note that selecting a gender will change the size options in `sizingData`.

Show size and width options (if applicable), and confirm consumer's selection

- Use the info from `sizingData` to display the available sizes, making note of the respective `questionId` and `answerId` values.
- Using the `questionId` and `answerId` values for the size selected by the consumer, call the `setSizeAnswer` method, like:

```
builderApi.setSizeAnswer('FUTUREELITEFA18:LTITEM8538:LTITEM8112:LTITEM8010:LTITEM403108','LTITEM8132','us-mens'))
```

This sets `isSelected: true` in `sizingData.answers`, indicating that a particular size was selected.

> **TIP**: Answering the gender and size-related questions are the only required Builder interactions for a design to be purchasable.

Save the Build

Save the Build

- Call the `saveDesign` method like:

```
builderApi.saveDesign()
```

This saves the current build configuration and returns a promise to send a metric ID for that build.

Show 'Add to Cart' CTA

Once you have a metric ID for the build, the consumer should be able to add their design to the shopping cart.

Show the consumer a way to add the product to their shopping cart with an 'Add to Cart' CTA.

- Display an 'Add to Cart' CTA that adds the design to the cart.
- Example button code using NCSS:

```
<button class="ncss-btn-primary-dark">Add to Bag</button>
```

Add to Bag

- This CTA should only be active once the gender, width, and size-related selections have been passed to the Builder, as shown in Show Gender, Width, and Size Options and Confirm Consumer's Selections.
- Once active, the specific behavior of this CTA can vary depending on your requirements, but here is an example:
  - Call the Carts API with the metric ID for the build to add the product to a cart.
  - Show an updated cart item count on the PDP and/or navigate the consumer to a cart page/view.

> **TIP**: For more see Adding Cart and Checkout to your Experience.

Contacting the Team

Slack	#nikeid-dev-systems
Confluence Space	NikeiD Systems Home
Team Contacts	Jason Mueller, Product Manager

Document Change Log

Summary	Date
Initial publish	06/17/2019

Next Steps

You've learned how to add Customization to your experience. Here are some next steps.

- Adding Cart & Checkout To Your Experience
- Using Nike APIs
- Glossary